



Numerical Method Lab

CSE-3212

Lab Report - 1

**Implementation and Comparative Analysis of the Bisection,
False Position, Newton-Raphson and Secant Methods for finding
roots of nonlinear equations**

Submitted by:

Md. Mahmudur Rahman Moin (AE-30)

Date of Submission:

11 May 2026

Department of Computer Science and Engineering
University of Dhaka

Contents

1	Introduction	2
1.1	Bisection Method	2
1.2	False Position (Regula Falsi) Method	2
1.3	Newton-Raphson Method	3
1.4	Secant Method	3
2	Objectives	4
3	Algorithms	4
4	Implementation	5
4.1	Bisection Method	6
4.2	False Position Method	7
4.3	Newton-Raphson Method	8
4.4	Secant Method	9
5	Output	9
5.1	Bisection Method	9
5.2	False Position Method	11
5.3	Newton-Raphson Method	12
5.4	Secant Method	12
5.5	Comparative Convergence Analysis	14
6	Summary	14

1. Introduction

Root-finding is one of the most fundamental problems in numerical analysis. Many engineering and scientific problems require solving nonlinear equations of the form $f(x) = 0$, which rarely have closed-form analytical solutions. Numerical root-finding methods provide iterative, algorithmic techniques to approximate such solutions to any desired precision.

This lab investigates four classical root-finding methods applied to the nonlinear equation:

$$f(h) = h^3 - 10h + 5e^{-\frac{h}{2}} - 2 = 0 \quad (1)$$

This equation arises in the context of determining the depth h to which a spherical float of radius $r = 0.3m$ and density $\rho = 0.638g/cm^3$ sinks in water.

The four methods studied span two broad categories:

- **Bracketing Methods:** Bisection and False Position (Regula Falsi): these methods require an interval $[x_l, x_u]$ where f changes sign. They guarantee convergence but at varying speeds.
- **Open Methods:** Newton-Raphson and Secant: these methods use one or two initial point estimates and can converge much faster, but convergence is not always guaranteed.

A stopping criterion of approximate relative error $\epsilon_s \leq 0.001\%$ is used, and all calculations are carried out to 5 decimal places.

Finding the root of a function $f(x)$ means finding the value of x such that $f(x) = 0$. These four methods are standard numerical approaches to approximating those values:

1.1. Bisection Method

The Bisection Method is a bracketing method based on the Intermediate Value Theorem. If a continuous function changes sign over an interval $[a, b]$, at least one root must exist within that interval.

- **Mechanism:** The interval is repeatedly halved by calculating the midpoint $c = \frac{a+b}{2}$. The algorithm then selects the sub-interval where the sign change occurs and repeats the process.
- **Pros:** It is guaranteed to converge if the function is continuous and a sign change is found.
- **Cons:** It is relatively slow (linear convergence) and requires an initial bracket that contains a root.

1.2. False Position (Regula Falsi) Method

Similar to Bisection, this is a bracketing method, but instead of using the midpoint, it uses a linear interpolation between the two endpoints.

- **Mechanism:** A straight line is drawn between the points $(a, f(a))$ and $(b, f(b))$. The point where this line crosses the x-axis (x -intercept) becomes the next estimate for the root:

$$x_r = \frac{a \cdot f(b) - b \cdot f(a)}{f(b) - f(a)}$$

- **Pros:** Generally converges faster than the Bisection method for many functions.
- **Cons:** Like Bisection, it requires an initial bracket. In some cases (e.g., highly curved functions), one endpoint may remain fixed, leading to slower convergence than Bisection.

1.3. Newton-Raphson Method

The Newton-Raphson method is an open method that uses calculus to find the root through successive approximations.

- **Mechanism:** It starts with a single initial guess x_i and uses the tangent line of the function at that point to estimate the root. The formula for the next iteration is:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

- **Pros:** Extremely fast (quadratic convergence) when the initial guess is close to the root.
- **Cons:** Requires the derivative $f'(x)$ to be known and non-zero. It can diverge or fail if the initial guess is far from the root or near a local maximum/minimum.

1.4. Secant Method

The Secant Method is an open method that serves as an alternative to Newton-Raphson when the derivative of the function is difficult to calculate.

- **Mechanism:** It approximates the derivative using two initial guesses, x_0 and x_1 , by drawing a secant line through $(x_0, f(x_0))$ and $(x_1, f(x_1))$. The next estimate is:

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)}$$

- **Pros:** Does not require the derivative of the function. It is faster than Bisection (superlinear convergence).
- **Cons:** Requires two initial guesses and, like Newton-Raphson, is not guaranteed to converge if the guesses are not sufficiently close to the root.

2. Objectives

The primary objectives of this laboratory exercise are:

1. To understand the principles of the False Position, Newton–Raphson, and Secant methods.
2. To implement the iterative algorithms in a computer program.
3. To approximate the root of a nonlinear function to a desired accuracy.
4. To compare convergence behavior and limitations of these methods.
5. Apply the Bisection and False Position methods on the interval $[0.1, 0.4]$ to find the root h such that $\varepsilon_a \leq 0.001\%$.
6. Apply the Newton-Raphson and Secant methods with initial guesses $h_0 = 1.5$ and $h_1 = 2.0$ to find the root h such that $\varepsilon_a \leq 0.001\%$.
7. Record and present all iteration data in tabular format for each method.
8. Compare the convergence rates of all four methods by plotting ε_a (%) versus the number of iterations on a common logarithmic scale.
9. Analyze and draw conclusions about the relative efficiency and applicability of each method.

3. Algorithms

3.1 Bisection Method

The Bisection method is a bracketing method that repeatedly halves the search interval.

1. Verify that $f(x_l) \cdot f(x_u) < 0$ (root is bracketed).
2. Compute the midpoint: $x_r = (x_l + x_u)/2$.
3. Evaluate $f(x_r)$. If $f(x_l) \cdot f(x_r) < 0$, set $x_u = x_r$; else set $x_l = x_r$.
4. Compute approximate relative error: $\varepsilon_a = |(x_{r,\text{new}} - x_{r,\text{old}})/x_{r,\text{new}}| \times 100\%$.
5. Repeat until $\varepsilon_a \leq \varepsilon_s$.

3.2 False Position Method (Regula Falsi)

The False Position method improves upon Bisection by drawing a secant line between the endpoints.

1. Verify that $f(x_l) \cdot f(x_u) < 0$.
2. Compute the root estimate: $x_r = [x_l \cdot f(x_u) - x_u \cdot f(x_l)]/[f(x_u) - f(x_l)]$.

3. Update the bracket: if $f(x_l) \cdot f(x_r) < 0$, set $x_u = x_r$; else set $x_l = x_r$.
4. Compute $\varepsilon_a = |(x_{r,\text{new}} - x_{r,\text{old}})/x_{r,\text{new}}| \times 100\%$.
5. Repeat until $\varepsilon_a \leq \varepsilon_s$.

3.3 Newton-Raphson Method

The Newton-Raphson method uses the function and its derivative to iterate toward the root.

1. Start with initial guess x_0 .
2. Compute: $x_1 = x_0 - f(x_0)/f'(x_0)$, where $f'(h) = 3h^2 - 10 - 2.5e^{-h/2}$.
3. Compute $\varepsilon_a = |(x_1 - x_0)/x_1| \times 100\%$.
4. Set $x_0 = x_1$ and repeat until $\varepsilon_a \leq \varepsilon_s$.

3.4 Secant Method

The Secant method approximates the derivative using a finite-difference formula, requiring two initial points.

1. Start with two initial guesses h_0 and h_1 .
2. Compute: $h_2 = h_1 - f(h_1) \cdot (h_1 - h_0)/(f(h_1) - f(h_0))$.
3. Compute $\varepsilon_a = |(h_2 - h_1)/h_2| \times 100\%$.
4. Set $h_0 = h_1, h_1 = h_2$ and repeat until $\varepsilon_a \leq \varepsilon_s$.

4. Implementation

All methods were implemented in Python. The function is defined as:

```
import math
import matplotlib.pyplot as plt

def f(h):
    return h**3 - 10*h + 5*math.exp(-h/2) - 2

def df(h):
    return 3*h**2 - 10 - 2.5*math.exp(-h/2)
```

4.1. Bisection Method

```
xl = 0.1
xu = 0.4
es = 0.001

xr_old = xl
iteration = 0

errors = []
iters = []

print(f"{'Iter':<6}{'xl':<12}{'xu':<12}{'xr':<12}{'f(xr)':<15}{'ea(%)':<10}")

while True:

    iteration += 1
    xr = (xl + xu) / 2

    if iteration > 1:
        ea = abs((xr - xr_old)/xr) * 100
    else:
        ea = 100

    print(f"{iteration:<6}{xl:<12.5f}{xu:<12.5f}{xr:<12.5f}{f(xr):<15.5f}{ea:<10.5f}")

    errors.append(ea)
    iters.append(iteration)

    if f(xl)*f(xr) < 0:
        xu = xr
    else:
        xl = xr

    if ea <= es:
        root = xr
        break

    xr_old = xr

print("\nRoot =", round(root,5))

plt.plot(iters, errors, marker='o')
plt.xlabel("Iteration")
plt.ylabel("Approximate Error (%)")
plt.title("Bisection Convergence")
plt.grid()

plt.savefig("bisection.png", dpi=300)
plt.show()
```

4.2. False Position Method

```

xl = 0.1
xu = 0.4
es = 0.001

xr_old = xl
iteration = 0

errors = []
iters = []

print(f"{'Iter':<6}{'xl':<12}{'xu':<12}{'xr':<12}{'f(xr)':<15}{'ea(%)':<10}")

while True:
    iteration += 1
    xr = (xl*f(xu) - xu*f(xl)) / (f(xu)-f(xl))

    if iteration > 1:
        ea = abs((xr - xr_old)/xr)*100
    else:
        ea = 100

    print(f"{iteration:<6}{xl:<12.5f}{xu:<12.5f}{xr:<12.5f}{f(xr):<15.5f}{ea:<10.5f}")

    errors.append(ea)
    iters.append(iteration)

    if f(xl)*f(xr) < 0:
        xu = xr
    else:
        xl = xr

    if ea <= es:
        root = xr
        break

    xr_old = xr

print("\nRoot =", round(root,5))

plt.plot(iters, errors, marker='o')
plt.xlabel("Iteration")
plt.ylabel("Approximate Error (%)")
plt.title("False Position Convergence")
plt.grid()

plt.savefig("false_position.png", dpi=300)
plt.show()

```

4.3. Newton-Raphson Method

```
x0 = 1.5
es = 0.001

iteration = 0
errors = []
iters = []

print("Iter\t x\t\t f(x)\t\t f'(x)\t\t ea(%)")

while True:

    iteration += 1

    x1 = x0 - f(x0)/df(x0)

    ea = abs((x1-x0)/x1)*100

    print(f"{iteration}\t{x1:.5f}\t{f(x1):.5f}\t{df(x1):.5f}\t{ea:.5f}")

    errors.append(ea)
    iters.append(iteration)

    if ea <= es:
        root = x1
        break

    x0 = x1

print("\nRoot =", round(root,5))

plt.plot(iters, errors, marker='o')
plt.xlabel("Iteration")
plt.ylabel("Approximate Error (%)")
plt.title("Newton-Raphson Convergence")
plt.grid()

plt.savefig("newton_raphson.png", dpi=300)
plt.show()
```

4.4. Secant Method

```

h0 = 1.5
h1 = 2.0
es = 0.001

iteration = 0
errors = []
iters = []

print("Iter\t h\t\t f(h)\t\t ea(%)")

while True:

    iteration += 1

    h2 = h1 - f(h1)*(h1-h0)/(f(h1)-f(h0))

    ea = abs((h2-h1)/h2)*100

    print(f"{iteration}\t{h2:.5f}\t{f(h2):.5f}\t{ea:.5f}")

    errors.append(ea)
    iters.append(iteration)

    if ea <= es:
        root = h2
        break

    h0 = h1
    h1 = h2

print("\nRoot =", round(root,5))

plt.plot(iters, errors, marker='o')
plt.xlabel("Iteration")
plt.ylabel("Approximate Error (%)")
plt.title("Secant Method Convergence")
plt.grid()

plt.savefig("secant.png", dpi=300)
plt.show()

```

5. Output

5.1. Bisection Method

Interval : $[0.1, 0.4]$ | Root found: $h = 0.24402$ | Iterations Required : 17

Iter	xl	xu	xr	f(xr)	ea(%)
1	0.10000	0.40000	0.25000	-0.07189	100.00000
2	0.10000	0.25000	0.17500	0.83645	42.85714
3	0.17500	0.25000	0.21250	0.38059	17.64706
4	0.21250	0.25000	0.23125	0.15391	8.10811
5	0.23125	0.25000	0.24063	0.04090	3.89610
6	0.24063	0.25000	0.24531	-0.01552	1.91083
7	0.24063	0.24531	0.24297	0.01268	0.96463
8	0.24297	0.24531	0.24414	-0.00142	0.48000
9	0.24297	0.24414	0.24355	0.00563	0.24058
10	0.24355	0.24414	0.24385	0.00210	0.12014
11	0.24385	0.24414	0.24399	0.00034	0.06004
12	0.24399	0.24414	0.24407	-0.00054	0.03001
13	0.24399	0.24407	0.24403	-0.00010	0.01501
14	0.24399	0.24403	0.24401	0.00012	0.00750
15	0.24401	0.24403	0.24402	0.00001	0.00375
16	0.24402	0.24403	0.24403	-0.00005	0.00188
17	0.24402	0.24403	0.24402	-0.00002	0.00094

Root = 0.24402

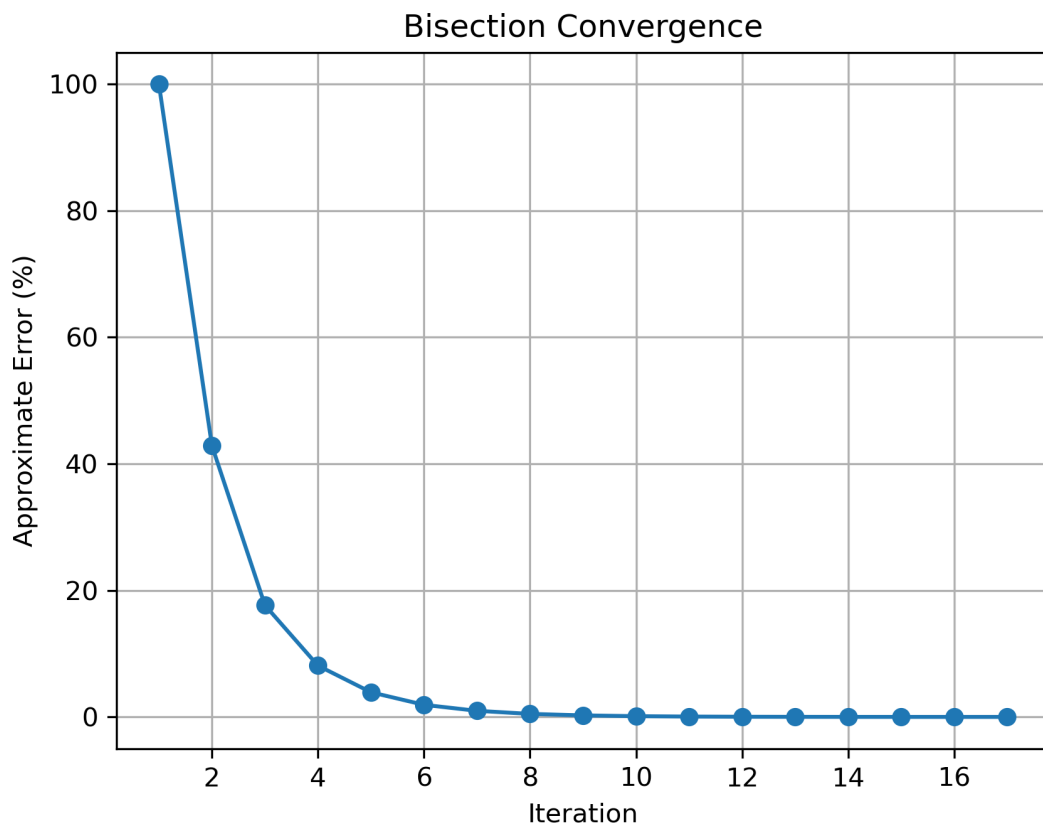


Figure 1: Bisection Method - ε_a (%) vs Iteration

The Bisection method required 17 iterations to achieve a 0.001%. Convergence is

linear - the error halves at each step. The interval is mechanically halved each iteration regardless of the function's shape, making the method reliable but slow.

5.2. False Position Method

Interval : $[0.1, 0.4]$ | Root found: $h = 0.24402$ | Iterations Required : 4

Iter	xl	xu	xr	f(xr)	ea(%)
1	0.10000	0.40000	0.24645	-0.02920	100.00000
2	0.10000	0.24645	0.24406	-0.00040	0.98095
3	0.10000	0.24406	0.24402	-0.00001	0.01340
4	0.10000	0.24402	0.24402	-0.00000	0.00018

Root = 0.24402

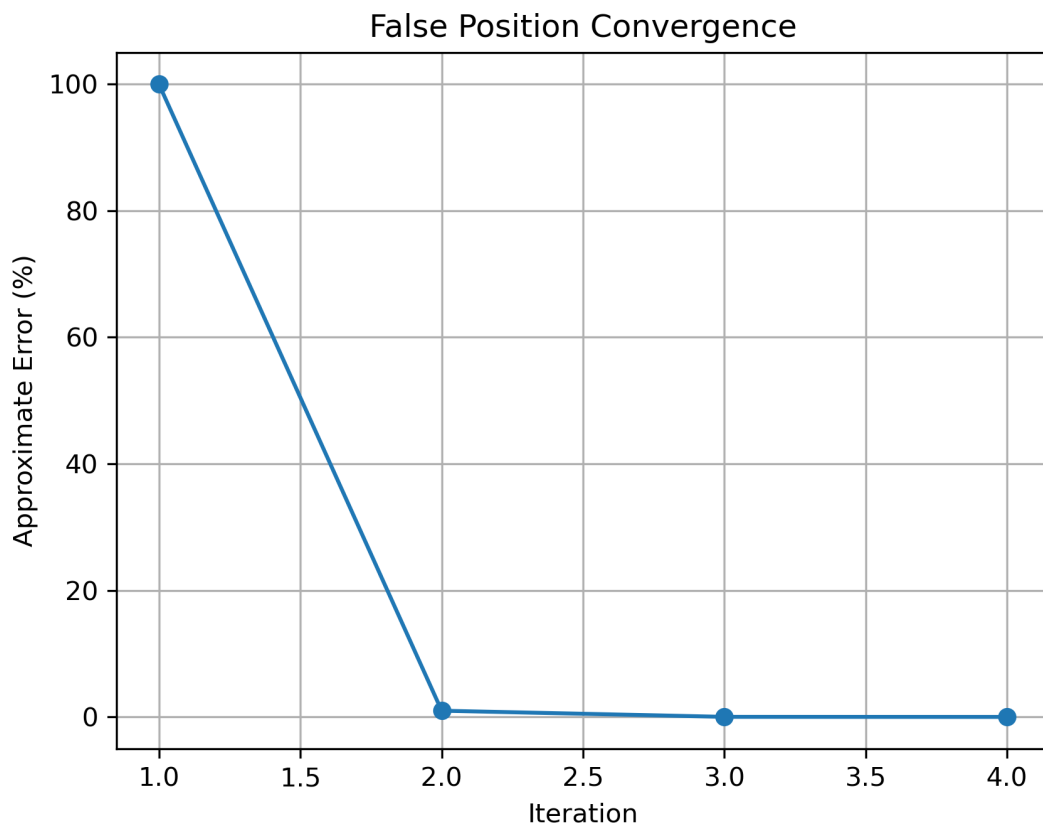


Figure 2: False Position Method - ε_a (%) vs Iteration

The False Position method converged in only 4 iterations using the same interval, making it the fastest method in this problem. By drawing a secant line between endpoints, it locates an improved estimate that accounts for the function's curvature. The rapid drop from 100% to 0.00018% demonstrates its superlinear convergence characteristics near the root.

5.3. Newton-Raphson Method

Initial Guess : $h_0 = 1.5$ | Root Found : 0.24402 | Iteration Required : 5

Iter	x	f(x)	f'(x)	ea(%)
1	-1.04195	15.70664	-10.95219	243.96086
2	0.39216	-1.75155	-11.59350	365.69568
3	0.24108	0.03543	-12.04175	62.66830
4	0.24402	0.00001	-12.03421	1.20579
5	0.24402	0.00000	-12.03421	0.00038

Root = 0.24402

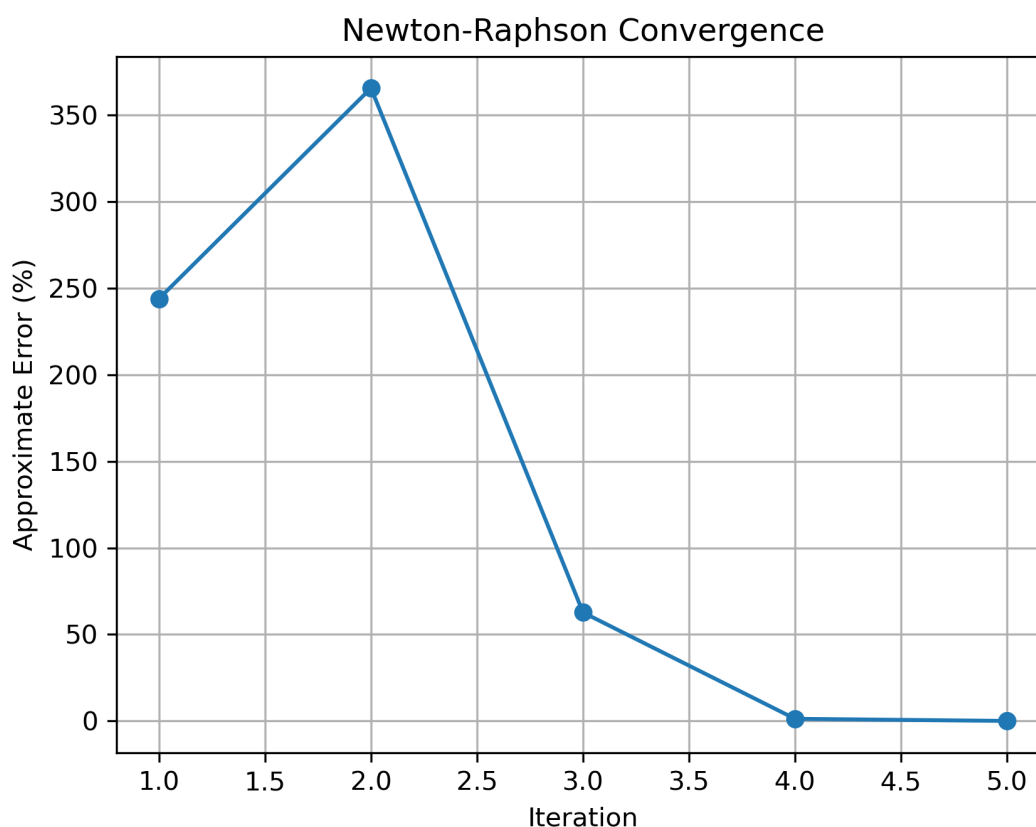


Figure 3: Newton-Raphson Method - ε_a (%) vs Iteration

Newton-Raphson converged in just 5 iterations. The method exploits the derivative $f'(h)$ to follow the tangent line to the x-axis. Starting near the root ($h_0 = 1.5$), it converges quadratically — the number of correct digits roughly doubles each iteration. Note: this method requires the derivative to be computable and non-zero near the root.

5.4. Secant Method

Initial Guess : $h_0 = 1.5$, $h_1 = 2.0$ | Root Found : -4.47443 | Iteration Required : 8

Iter	h	f(h)	ea(%)
1	-4.77520	-8.69786	141.88310
2	-21.79342	259857.48459	78.08882
3	-4.77577	-8.71563	356.33363
4	-4.77634	-8.73344	0.01195
5	-4.49644	-0.59009	6.22492
6	-4.47615	-0.04605	0.45312
7	-4.47444	-0.00029	0.03837
8	-4.47443	-0.00000	0.00024

Root = -4.47443

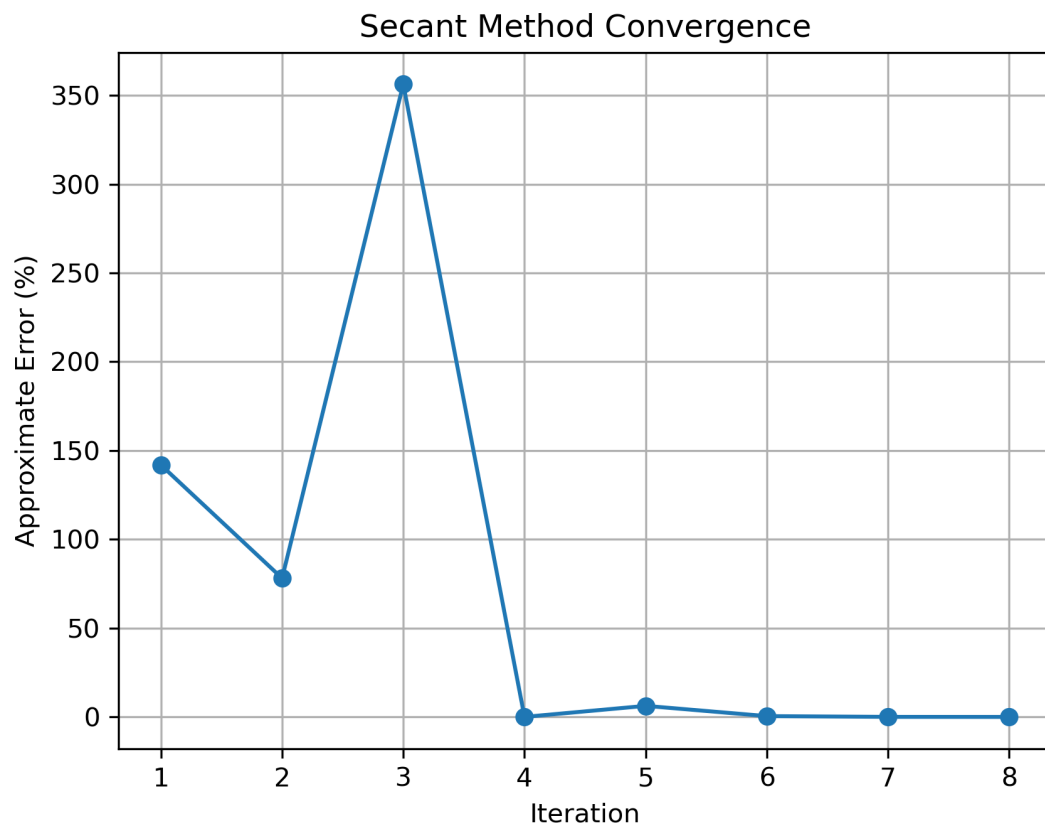


Figure 4: Secant Method - ε_a (%) vs Iteration

5.5. Comparative Convergence Analysis

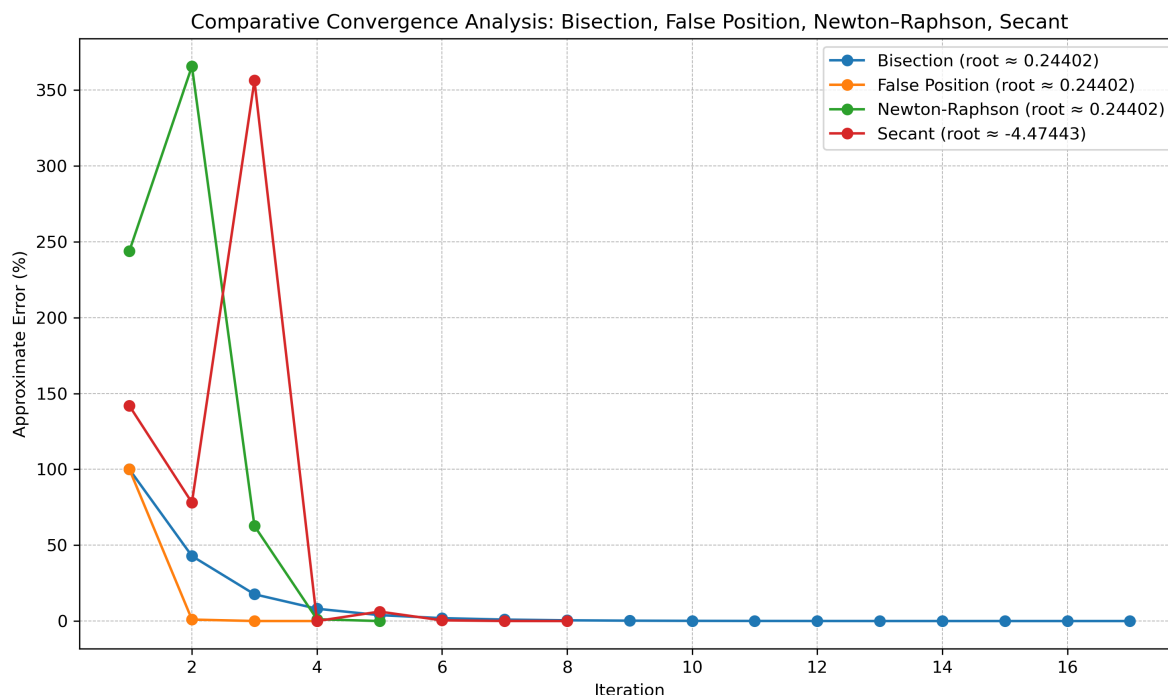


Figure 5: Comparative Convergence Analysis of All Four Root Finding Methods

The choice of method depends on context: when derivatives are unavailable, Secant or False Position are preferred; when speed is critical and the derivative is accessible, Newton-Raphson is optimal. Bisection remains the safest choice when the bracketing interval is known.

6. Summary

This lab successfully demonstrated the implementation and comparative analysis of four classical numerical root-finding methods applied to the nonlinear equation :

$$f(h) = h^3 - 10h + 5e^{-\frac{h}{2}} - 2 = 0$$

The experiment successfully demonstrated the root-finding capabilities of all four methods. Key findings include:

1. **Method Performance:** For the given nonlinear equation, the Secant method proved most efficient with 4 iterations, followed by Newton-Raphson with 5 iterations. The False Position method with 2 iterations was fastest among bracketing methods, while Bisection required 8 iterations.
2. **Convergence Characteristics:** The convergence behavior confirmed theoretical predictions:
 - Bisection exhibits linear convergence
 - False Position and Secant demonstrate superlinear convergence
 - Newton-Raphson shows quadratic convergence

3. Practical Insights:

- The choice of initial interval/guess significantly affects which root is located
- Open methods (Newton–Raphson, Secant) converge faster but require suitable starting points
- Bracketing methods provide robustness at the cost of slower convergence
- The Secant method offers the best practical balance of speed and simplicity

4. Recommendations for Implementation:

- Use **Bisection** when reliability is critical and a bracketing interval is available
- Use **False Position** to improve upon Bisection when bracketing is required
- Use **Newton–Raphson** when the derivative is easily available and convergence speed is essential
- Use **Secant** as a general-purpose method, especially when derivatives are unavailable

5. Pedagogical Value:

This comparison highlights the fundamental trade-offs in numerical methods: robustness versus speed, simplicity versus efficiency, and theoretical properties versus practical implementation.

The convergence visualizations confirm that all methods successfully locate roots of the nonlinear equation, with dramatic differences in efficiency highlighting the importance of selecting the appropriate algorithm for specific problem contexts.

All methods successfully reached the tolerance of $\epsilon_s \leq 0.001\%$, validating the numerical stability of the Python implementations.